A learned-correction value function with turn-level lookahead for 2-player Catan

Team TFI · Catan agent hackathon, IIT Bombay, 17 Oct 2026

1. Problem and constraints

Our agent plays 2-player, 10-VP Catan without player trading on the organisers' catanatron build. It plays against F (catanatron's one-ply `ValueFunctionPlayer`), O (a trained bot) and other teams. Three facts shaped every design choice.

- **Compute is tiny.** The clock is 300 ms plus 12.5 ms per decision, capped at 1.5 s per game. We measured a `copy()+execute()` at about 25 µs and F's evaluation at 65 µs, so a decision can afford roughly 150 positions. MCTS and depth-3 alpha-beta run out of clock in every game (problem statement, Section 7).
- **Hidden information is small.** The opponent's resource hand is exactly derivable; only their unplayed development cards are hidden. Because of this we sample one world per decision (perfect-information sampling, Long et al. 2010) and spend compute on other things.
- **F's evaluation is a tiered linear sum.** Public VP is weighted 3×10^{14}, production 10^8, reachable production 10^4, building spots 10^3, and hand and dev-card terms 1 to 100. This works almost like a lexicographic order: a term only matters between moves that tie on every higher tier.

2. Method

2.1 Fast exact re-implementation of F's evaluation

We rewrote `base_fn` to compute each feature once (F's version computes production features twice) and only for our own seat. It is bit-for-bit identical to `base_fn` on 1,740 random positions and 3× faster (22 vs 65 µs), and that speed buys the search below. With lookahead off, our agent reproduces F's results exactly (193/400 wins on the same seeds as F vs F).

2.2 Learned win model as a correction term (the main gain)

We recorded positions after every action, from both seats, in 16,000 F-vs-F games (1.04M positions), each labelled with the eventual winner. On these we fit a logistic regression over 50 features. For each player the features cover public VP, production per resource, resource variety, road-reachable production, open building spots, hand composition and size, longest-road length, knights played, dev cards held, settlements, cities and ports, plus whose turn it is and our hidden VP cards. The model predicts the eventual winner for 74% of held-out positions (split by game): 62% in turns 0–20 and 80% from turn 40.

Used on its own as the evaluation, the model is poor (26% vs F with one-ply). It cannot judge within-turn trade-offs such as spending a hand versus holding it, which F's hand-set terms do encode. We therefore use a **hybrid**: $V(s) = V_F(s) + \alpha\cdot\text{logit}\, p_{\text{win}}(s)$ with $\alpha = 10^6$. At that scale the learned term sits between F's production tier and its lower tiers. It never overrides a VP or a large production change, but it decides the many moves F treats as near ties: roads, bank trades, dev-card purchases, robber placement and ending the turn. Larger α (10^8) lets the model overrule production and loses to F (47%).

2.3 Same-turn lookahead

F is myopic inside its own turn. A bank trade that enables a settlement looks bad on its own, because the trade alone lowers the hand terms. For the top-4 moves by one-ply value we roll the rest of our turn forward greedily (up to 4 more own moves) and score each move by the best value reached before ending the turn. This cannot be fooled by the opponent, because the rollout stops when the turn passes.

2.4 Opening simulation

For each initial-settlement decision we take the 6 best candidates by one-ply value and simulate the rest of the placement phase. Both seats place greedily by our evaluation from their own side. We keep the candidate whose finished placement scores best. The 300 ms starting bank pays for this.

2.5 Time manager and safety

The per-decision budget is min(cap, 0.25·(cpu_left − 200 ms)), with a 150 ms cap in the opening. Below 200 ms the agent drops to one-ply, below 40 ms to one-ply with F's cheap evaluation, and below 15 ms to a fixed move-type priority with no copying. Every decision is wrapped: on any exception it falls back to one-ply, then to index 0. On the scoring machine v2 used 368 ms of CPU per game (1,029 ms worst), with 0 games over the clock and 0 faults.

2.6 In-run opponent adaptation

One agent process plays a whole match against a single opponent, and adapting is allowed. The final agent therefore holds several complete strategies ("arms") that are equally strong against F but play differently: the linear model, an MLP model, and an MLP opening judge. Before each game it picks an arm by Thompson sampling from a Beta posterior over that arm's wins. It labels its own games: a game counts as won when our chosen move produced the 10th VP. *[Result: to fill in from the bandit runs.]*

3. Experimental protocol

All local numbers come from the kit's own harness (`catan_kit.match`), run in parallel, with each board played from both seats with the same dice. Tuning used dev boards (seed 990000), and every claimed gain was re-checked on separate holdout boards (seed 700000). We needed ≥1,000 games per arm (95% CI ±3 pp) before accepting a change. Weight tuning used its own boards, and model training used separate games. Every run is logged in `tools/results.csv`, 60+ runs in all.

4. Results

Agent (vs F)	Dev (1000)	Holdout (1000)	CPU/game
F vs F (sanity)	0.483 (400)		223 ms
F evaluation + same-turn lookahead	0.517		419 ms
Hybrid, one-ply	0.573 (400)		222 ms
Hybrid + lookahead (v1)	0.643 (400)	0.622	346 ms
Hybrid + lookahead, no opening simulation	0.642	0.640	361 ms
v2 = hybrid + lookahead + opening simulation	**0.684**	**0.672**	455 ms
v2 without the learned term (ablation)	0.517		419 ms
v2 without lookahead (ablation)	0.600		351 ms

Table 1. Win rate against F (95% CI ±0.03 at 1,000 games, ±0.05 at 400). CPU is on a laptop, where F itself uses 186–223 ms against 158 ms on the scoring machine. The learned term is worth 16.7 pp, lookahead 8.4 pp and the opening simulation 4.2 pp. The parts interact: lookahead alone adds only 3 pp to F, but 7 pp to the hybrid.

Scoring machine (public boards). v2: 0.649 ± 0.030 vs F and 0.487 ± 0.031 vs O, 368 ms of CPU per game, 0 games over the clock, 0 faults. *[v3 / v4: to fill in.]* The team machine (Linux, official script) gave 0.700 ± 0.064 over 200 games. The 3–4 pp drop from local to public is within the board-set variance we observed between our own dev and holdout sets.

How v2 wins. Over 300 games vs F, v2 ends with longest road in 69% of games against 30% for F, and largest army in 28% against 3%. It buys 3.3 dev cards per game against 0.9, while building slightly *fewer* settlements and cities (4.0 vs 4.5). Longest road decides games: v2 holds it in 92% of its wins but only 31% of its losses. The learned term moved play towards the two 2-VP awards that F's tiered weights almost ignore. We never hand-coded this.

Robustness. Other teams will mostly field F-like agents, so we built five F variants with re-tiered weights: build-heavy, dev/army-heavy, road-heavy, city-saving, and catanatron's tuned "contender". Against them v2 scores 0.685, 0.705, 0.915, 0.693 and 0.675 (600 games each). Its worst half (contender, F, build) averages 0.681, the same as against F itself. It beats v1 head-to-head 0.546 ± 0.031.

5. What did not work, and why that matters

Attempt	vs F	Lesson
catanatron AlphaBeta, depth 2 (expectiminimax on F's evaluation), no clock	0.410	Deeper search does not fix a weak evaluation
Wider/deeper lookahead (top 8, depth 7); opening with 10–16 candidates	0.63–0.69	Search saturates quickly
Chance moves averaged over 4 samples	0.658–0.663	More CPU, no gain
Retraining on our own games (DAgger-style)	0.562	Feedback loop (below)
MLP win model (val log-loss 0.472 vs 0.491 linear)	0.667–0.678	Better prediction did not mean better play
Hand-designed strategy terms: army race, zero-sum VP, road race, blocking, hidden-VP valuation	0.50–0.69	Neutral or harmful
Dedicated robber (exact steal expectation) and discard logic	0.663–0.679	Neutral
Opening model trained on 32k end-of-placement positions	0.644–0.693	No gain on holdout
CEM weight search, 200–240 games per candidate	flat	Inconclusive at ±7 pp noise per candidate

- **The evaluation, not search depth, is the bottleneck under a hard compute cap.** Expectiminimax with F's evaluation loses to one-ply F. A small correction term from outcome data gave our biggest gain.
- **Learned corrections should be scaled into the gaps of a hand-built order, not replace it.** With α too large the model overrules production and play gets worse. With α at about 10^6 it settles the moves the hand-built order leaves tied.
- **Training on your own games can teach the model to recognise the player, not the position.** In our own games our agent wins 65%, so features that mark its play style (dev-card buying, knights) predict wins. Retraining on those games made the agent worse. Controlling for player identity during training is the principled fix.
- **Hand-coded strategy hurt once a learned term was present.** The model already prices the army and road races; adding them again double-counts.
- **Noise discipline.** Several apparent +2–3 pp gains on 400–600 games vanished on 1,000 holdout games. We report only changes confirmed on both board sets.

References and public material used

- B. Collazo, *catanatron* (engine, F, AlphaBeta), github.com/bcollazo/catanatron, GPL-3.0; the organisers' patched build and `catan_kit`.
- M. Buro, Improving heuristic mini-max search by supervised learning, *Artificial Intelligence*, 2002.
- G. Tesauro, Temporal difference learning and TD-Gammon, *CACM*, 1995 (afterstate evaluation).
- S. Ross, G. Gordon, J. Bagnell, A reduction of imitation learning to no-regret online learning (DAgger), AISTATS 2011.
- J. Long, N. Sturtevant, M. Buro, T. Furtak, Understanding the success of perfect information Monte Carlo sampling, AAAI 2010.
- B. Ballard, The *-minimax search procedure for trees containing chance nodes, *Artificial Intelligence*, 1983.
- W. Thompson, On the likelihood that one unknown probability exceeds another, *Biometrika*, 1933 (Thompson sampling).
- R. Rubinstein, The cross-entropy method for combinatorial and continuous optimization, 1999.
- Libraries: numpy, scikit-learn (training only). Code written with the help of an LLM coding assistant (Claude), as the rules permit. All numbers come from our own runs.